Project Design Phase-II
Technology Stack (Architecture & Stack)

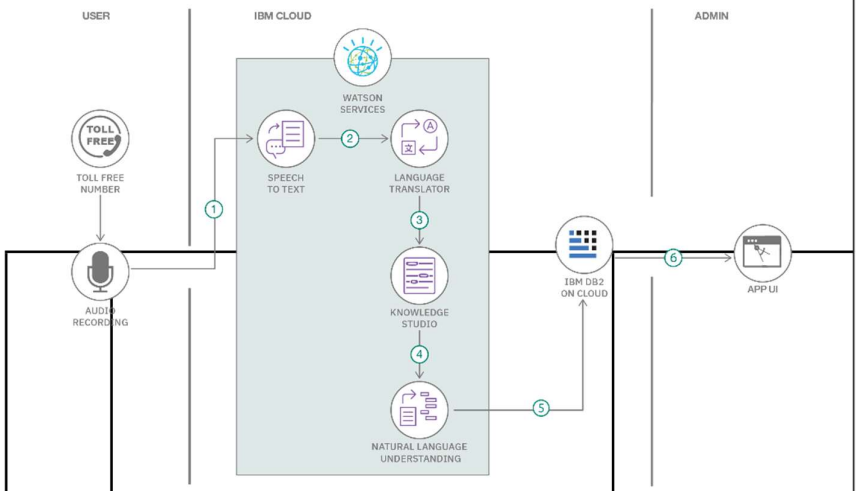
Date	2 March 2026
Team ID	
Project Name	PlantCare AI: Advanced Plant Disease Detection Using Transfer Learning
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Order processing during pandemics for offline mode

Reference: https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/



Guidelines:

- Include all the processes (As an application logic / Technology Block)
- Provide infrastructural demarcation (Local / Cloud)
- Indicate external interfaces (third party API's etc.)
- Indicate Data Storage components / services
- Indicate interface to machine learning models (if applicable)

S.No	Component	Description	Technology
1.	User Interface	Mobile App for plant image capture and disease diagnosis results	React Native (iOS/Android), Expo framework
2.	Application Logic-1	Image preprocessing and augmentation pipeline	Python 3.9, OpenCV, Pillow, NumPy
3.	Application Logic-2	Transfer Learning Model Inference (Plant Disease Classification)	TensorFlow 2.x, Keras, MobileNetV2 (pre-trained), EfficientNet
4.	Application Logic-3	Disease treatment recommendation engine	Python/Flask, Custom rule-based algorithm
5.	Database	User data, plant diagnosis history, treatment information	MongoDB (NoSQL) for flexible schema, PostgreSQL for structured data
6.	Cloud Database	Managed database service for scalability	MongoDB Atlas / AWS RDS
7.	File Storage	Plant images uploaded by users, model artifacts	AWS S3 / IBM Cloud Object Storage
8.	External API-1	Weather data for disease prediction based on environmental conditions	OpenWeatherMap API

9.	External API-2	Plant species identification API (for unknown plants)	Plant.id API / Pl@ntNet API
10.	Machine Learning Model	Transfer Learning model for plant disease detection	Custom-trained CNN (based on MobileNetV2/EfficientNet) trained on PlantVillage dataset and augmented data
11.	Infrastructure (Server / Cloud)	Application Deployment on Local System / Cloud	Hybrid: Local processing on mobile for basic inference, Cloud (AWS/GCP/IBM Cloud) for complex model inference, Kubernetes for model serving

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	All core technologies are open-source	TensorFlow, Keras, React Native, Flask, OpenCV, MongoDB, NumPy, Python
2.	Security Implementations	User authentication, data encryption, secure API communication	JWT tokens, HTTPS/TLS, bcrypt for password hashing, AWS IAM roles, API rate limiting, OWASP security practices
3.	Scalable Architecture	Microservices-based architecture with containerization for horizontal scaling	Docker containers, Kubernetes orchestration, React Native client, Flask/FastAPI microservices, Auto-scaling groups
4.	Availability	High availability through distributed deployment and redundancy	Load balancers (AWS ELB), multiple availability zones, CDN for static content, auto-healing Kubernetes pods

S.No	Characteristics	Description	Technology
5.	Performance	Optimized for real-time image processing and fast inference	- Response time: < 2 seconds per inference - Caching: Redis for frequent queries - CDN: CloudFront for image delivery

References:

- https://www.tensorflow.org/tutorials/images/transfer_learning
- https://reactnative.dev/
- https://www.tensorflow.org/lite
- https://kubernetes.io/